Week 5, Lesson 5

Advanced Networking Services

Reading: Chapter 3 – Amazon Networking Services (Doane & Seward, 2019, pp. 72–78)

Outline

1. VPC firewalls and VPC endpoints
2. Amazon Direct Connect
3. DNS with Route 53 and Amazon's CDN

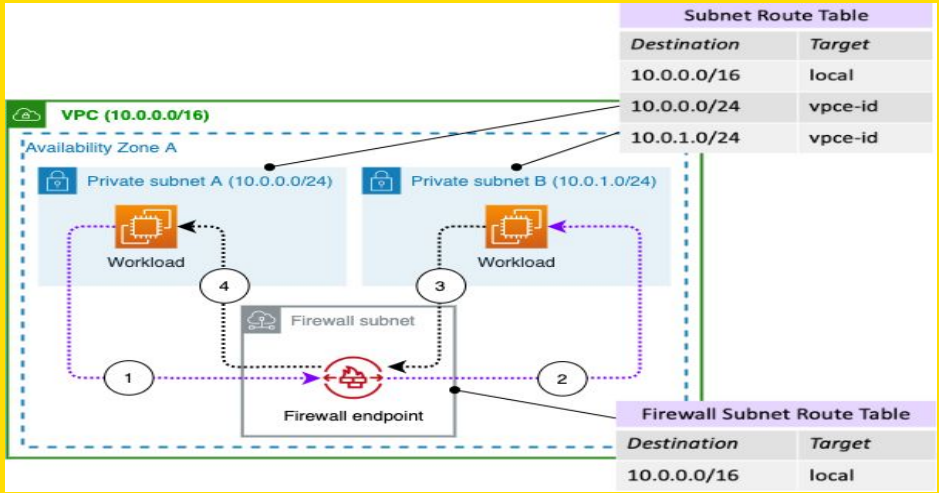
Dr. Rohit Kumar Kasera
Lecturer

VPC Firewalls and VPC Endpoints

VPC firewalls

VPC firewalls

- Security Groups act like a virtual firewall for EC2 instances.
- It control inbound and outbound traffic
- SG is stateful
- Allow/deny based on:
  - IP address
  - Port number (e.g., 22 for SSH)
  - Protocol (TCP, UDP)
- So, if inbound traffic is allowed, the response is automatically allowed back.



The VPC has two private subnets:
Private subnet A (10.0.0.0/24)

- Private subnet B (10.0.1.0/24)
- A dedicated firewall subnet contains the firewall endpoint.

How traffic moves

- A workload in subnet A sends traffic.
- The subnet route table sends that traffic to the firewall endpoint (vpce-id).
- The firewall checks the traffic against security rules.
- If allowed, traffic continues to subnet B (or elsewhere in the VPC).

Key idea

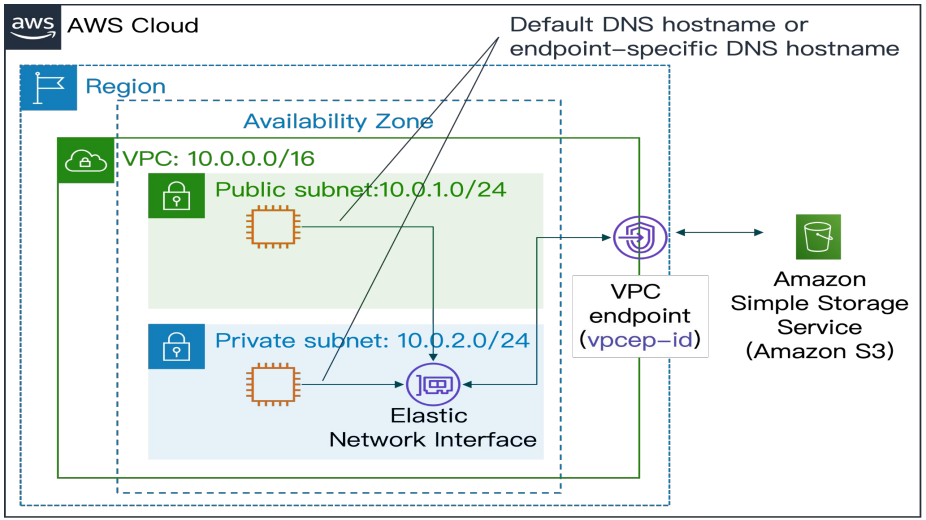
Instead of workloads talking directly to each other, traffic is forced through the firewall first for inspection and filtering.

Why this is useful

- Centralized security
- Blocks unwanted traffic
- Inspects east-west traffic between subnets
- Easier compliance and monitoring

Analogy: “All cars between neighborhoods must pass through a security checkpoint before entering another neighborhood.”

VPC endpoints



Public Subnet Route Table

Destination	Target
10.0.0.0/16	local
Amazon S3 ID	vpcep-id

Two types of endpoints:

- Interface endpoints (powered by AWS PrivateLink)
- Gateway endpoints (Amazon S3 and Amazon DynamoDB)

A VPC endpoint lets resources inside your VPC connect to AWS services (like Amazon S3) privately, without using the internet.

What happens

An EC2 instance inside the VPC wants to access S3.
Instead of going through:

- Internet Gateway
- NAT Gateway
- Public internet

it uses the **VPC endpoint**.

- AWS creates an **Elastic Network Interface (ENI)** inside your subnet.
- Traffic stays inside the AWS network securely.

Key benefit

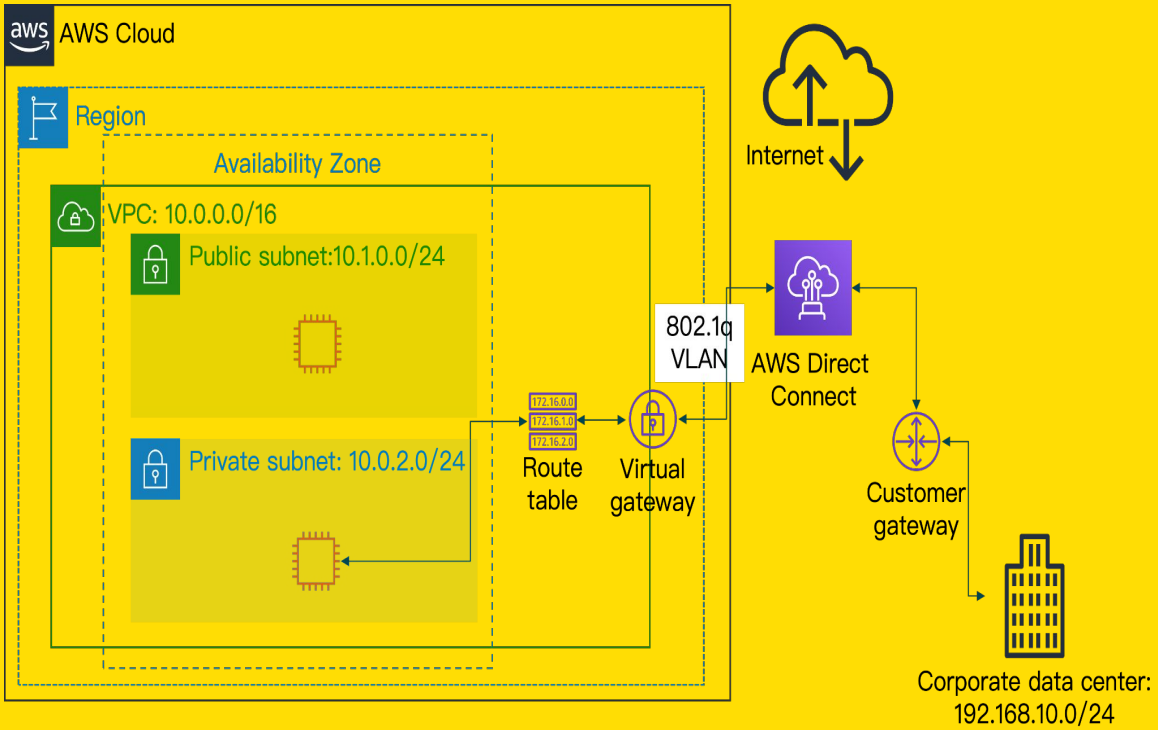
- Private and secure access to AWS services
- No public IP needed
- Lower exposure to internet threats

Analogy

- A VPC endpoint is like a private tunnel from your VPC directly to an AWS service, avoiding public roads (internet).

Amazon Direct Connect

- AWS Direct Connect enables a direct, dedicated connection between your internal network and AWS, bypassing internet service providers for a faster, more secure link.
- This setup uses a high-speed Ethernet fiber-optic cable which connects directly from your network's router to an AWS Direct Connect router at an AWS location.



- Think of AWS as a big office building.
- Internet connection = using normal public roads with traffic, delays, and shared access.
- AWS Direct Connect = having your own private highway directly from your office to AWS.

It means with Direct Connect:

- Your data travels on a dedicated private connection
- More stable and predictable speed
- Lower latency (faster response)
- More secure than public internet routes

For example: A bank connecting its on-premises data center to AWS may use Direct Connect so sensitive data travels through a private line instead of the internet.

Set up AWS Direct Connect

EIU

- **Review the prerequisites:** Before you begin, make sure you have an AWS account and access to the AWS Management Console, a Direct Connect location, and a device that can connect to the Direct Connect location.
- **Request a connection:** Log in to the AWS Management Console and go to the Direct Connect console.
    - Choose a Direct Connect location and request a new connection.
    - Provide details such as your connection name, port speed, and virtual private gateway.
- **Choose a Partner:** Choose a Direct Connect Partner who can provide a network connection between your on-premises environment and the Direct Connect location.
- **Configure the connection:** Once the connection has been approved by the Direct Connect Partner, you'll receive a Letter of Authorization and Connecting Facility Assignment (LOACFA) document.
    - You'll need to provide this document to the colocation facility where your equipment will be located.
    - You'll also need to configure your routers to connect to the Direct Connect location.
- **Test the connection:** After the connection is set up, test it to make sure it's working properly.
    - You can do this by creating a test VPC and connecting to it from your on-premises environment.
- **Start using Direct Connect:** Once you're confident that the connection is working as expected, you can start using Direct Connect to connect to your AWS resources.

Benefits of Using AWS Direct Connect

- It provides a connection with high bandwidth and low latency, hence reliability and performance are increased.
- Since a dedicated private connection is established completely bypassing the public internet hence adds to the security of your connection.
- Data transfer over AWS Direct Connect is very cost-effective as compared to using the typical public network.

Monitoring Tools

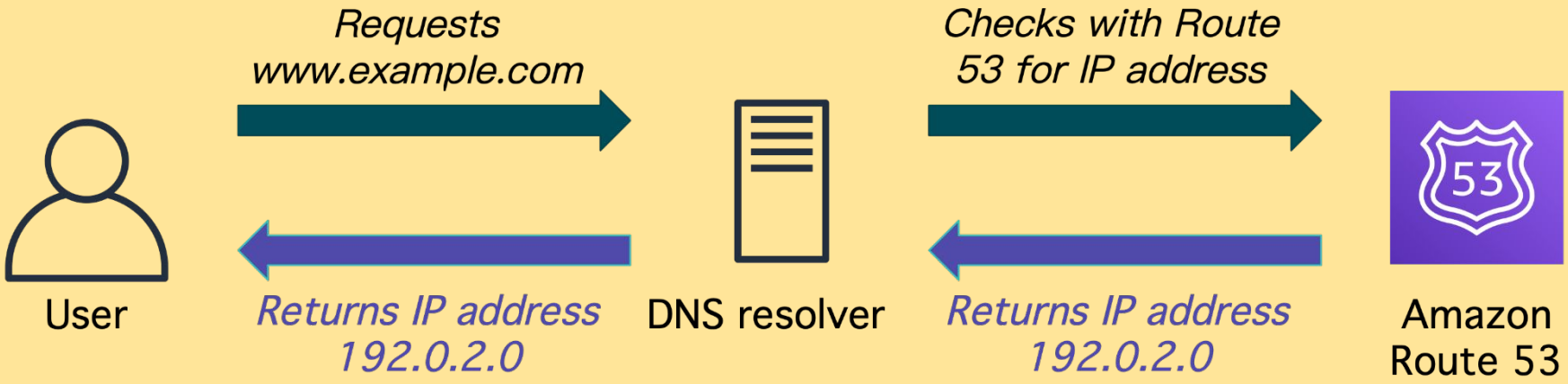
To ensure optimal performance, AWS offers several monitoring tools for Direct Connect:

- **Tagging**: Apply key-value tags to organize and manage your Direct Connect resources.
- **CloudTrail Integration**: Track all Direct Connect API calls as events for auditing and troubleshooting.
- **CloudWatch Metrics and Alarms**: Monitor key metrics like bandwidth usage and latency, and set up alerts for any unusual activity.

Understanding Amazon DNS – Route 53

Amazon Route 53

- Is a highly available and scalable Domain Name System (DNS) web service
- Is used to route end users to internet applications by translating names (like www.example.com) into numeric IP addresses (like 192.0.2.1) that computers use to connect to each other
- Is fully compliant with IPv4 and IPv6
- Connects user requests to infrastructure running in AWS and also outside of AWS
- Is used to check the health of your resources
- Features traffic flow
- Enables you to register domain names



Amazon Route 53 routing policies

Analogy

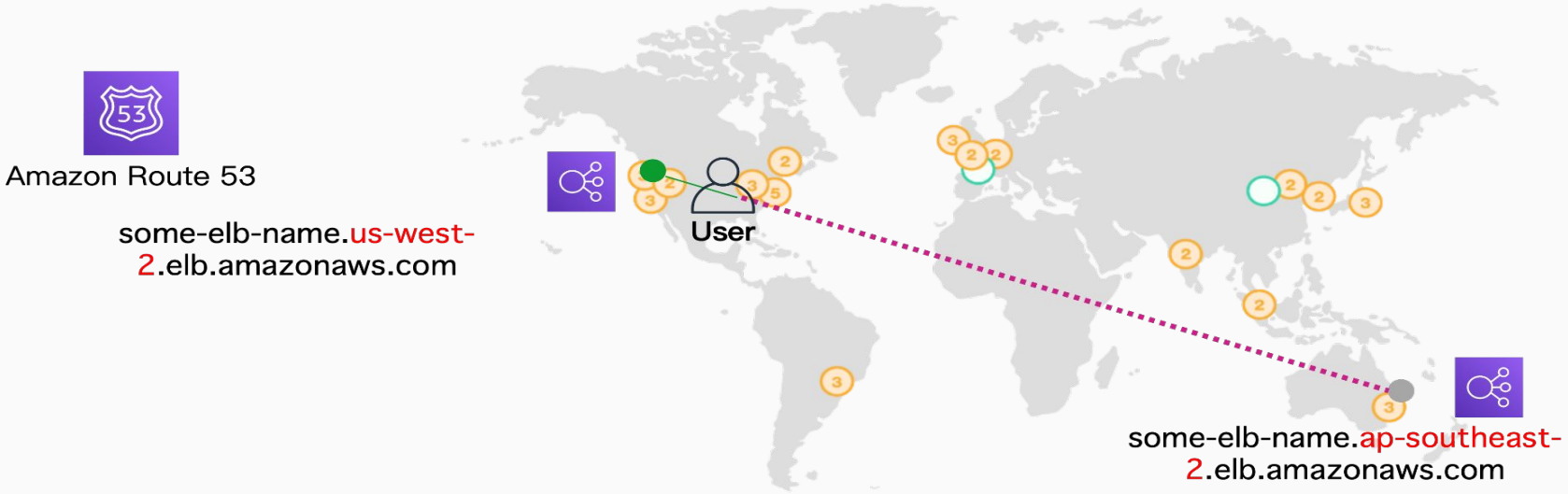
Think of the internet as a huge city.

- Websites are buildings
- IP addresses are exact map coordinates
- Domain names are building names
- Route 53 is the GPS/navigation system

Instead of remembering complicated coordinates, users simply enter a name, and Route 53 guides them to the correct location.

- **Simple routing** – Use in single-server environments
- Weighted round robin routing – Assign weights to resource record sets to specify the frequency
- **Latency routing** – Help improve your global applications
- **Geolocation routing** – Route traffic based on location of your users
- **Geoproximity routing** – Route traffic based on location of your resources
- **Failover routing** – Fail over to a backup site if your primary site becomes unreachable
- **Multivalue answer routing** – Respond to DNS queries with up to eight healthy records selected at random

Use case: Multi-region deployment



Name	Type	Value
example.com	ALIAS	some-elb-name.us-west-2.elb.amazonaws.com
example.com	ALIAS	some-elb-name.ap-southeast-2.elb.amazonaws.com

When a user opens the application, Route 53 checks:

> "Which region gives the fastest response time to this user?"

Then Route 53 sends the user to the nearest/lowest-latency region.

In this diagram

- User is in North America
- AWS measures network latency
- us-west gives faster response than ap-southeast
- So Route 53 routes the user to us-west

The numbered circles represent latency measurements from different AWS edge locations around the world.

Simple analogy

Imagine two pizza stores:

- One is 5 minutes away
- Another is 40 minutes away

When you order pizza, the system automatically chooses the closest store so delivery is faster.

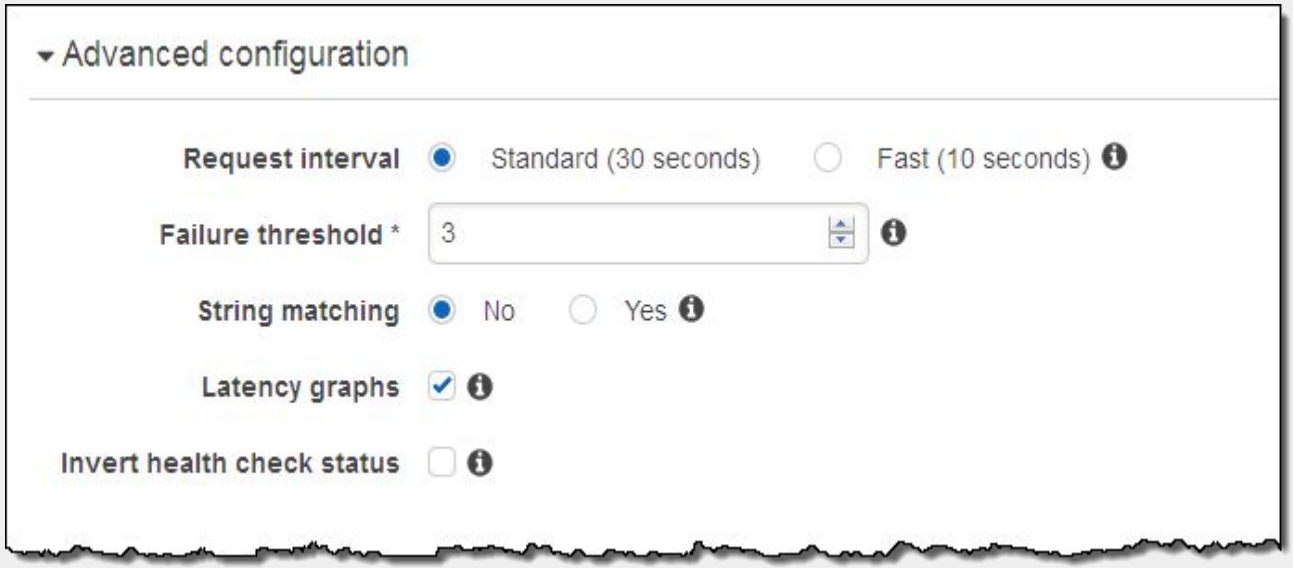
Route 53 does the same thing with AWS regions:

- It sends users to the closest/fastest server location.

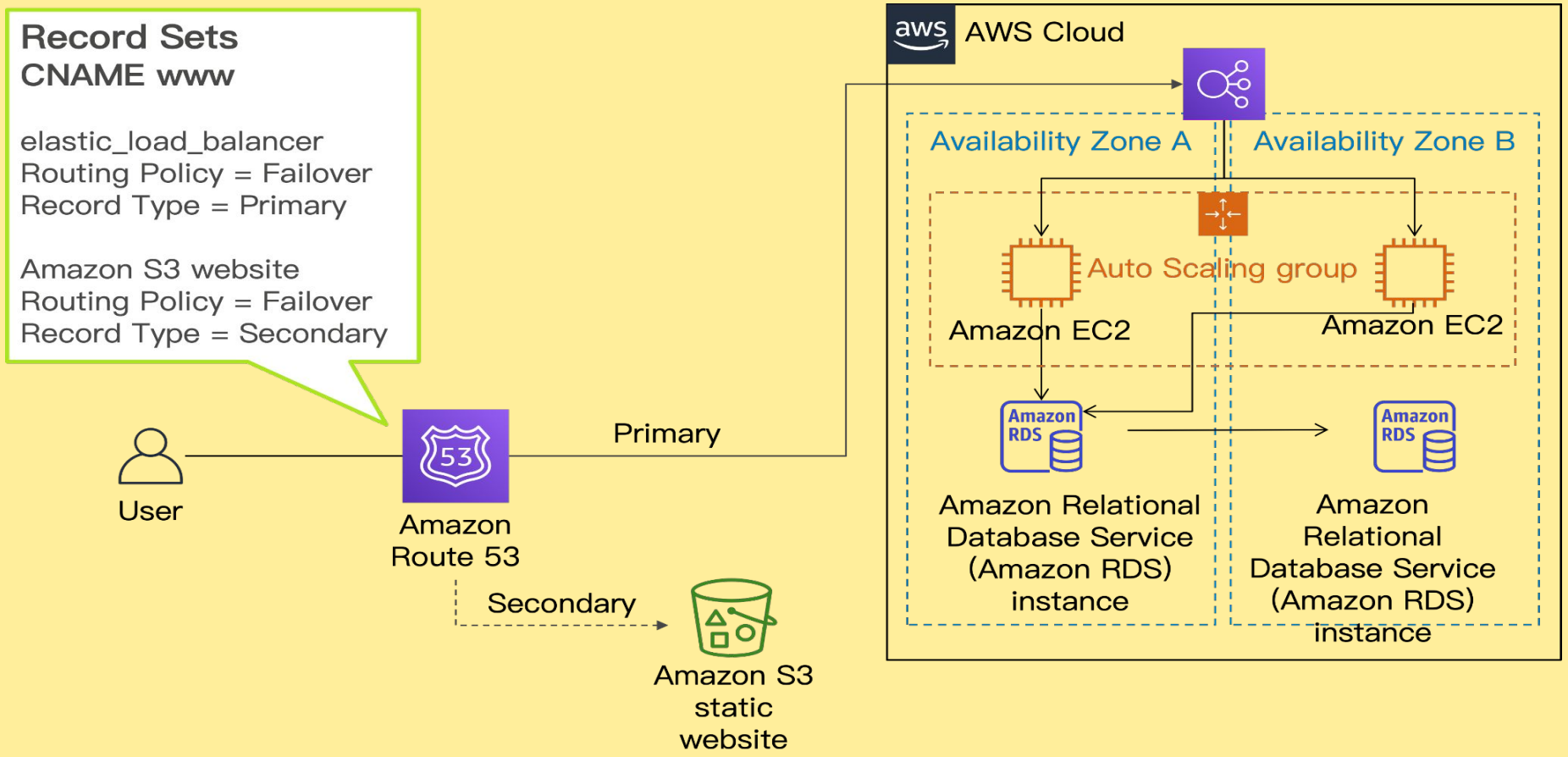
Amazon Route 53 DNS failover

Improve the availability of your applications that run on AWS by:

- Configuring backup and failover scenarios for your own applications
- Enabling highly available multi-region architectures on AWS
- Creating health checks



DNS failover for a multi-tiered web application



Explanations

The setup has:

- A main application running on:
  - Load Balancer
  - Auto Scaling EC2 instances
  - Amazon RDS
- A backup static website hosted on:
  - Amazon S3

Route 53 configuration

There are two DNS records:

1. **Primary record**
   - Points to the main application (Elastic Load Balancer)
2. **Secondary record**
   - Points to the S3 static website

How failover works

- Normally, users go to the main application.
- Route 53 continuously checks health/status.
- If the main application fails:
  - Route 53 automatically redirects users to the backup S3 website.

Why this is useful

- Keeps the website available during outages
- Improves reliability
- Provides disaster recovery

Simple analogy

Think of a restaurant:

- Main dining hall = primary application
- Backup takeaway counter = S3 backup site

If the dining hall closes suddenly, customers are automatically redirected to the takeaway counter so service continues.

Content delivery network (CDN)

- Is a globally distributed system of caching servers
- Caches copies of commonly requested files (static content)
- Delivers a local copy of the requested content from a nearby cache edge or Point of Presence
- Accelerates delivery of dynamic content
- Improves application performance and scaling
- Fast, global, and secure CDN service
- Global network of edge locations and Regional edge caches
- Self-service model
- Pay-as-you-go pricing

CloudFront infrastructure:

Key components:

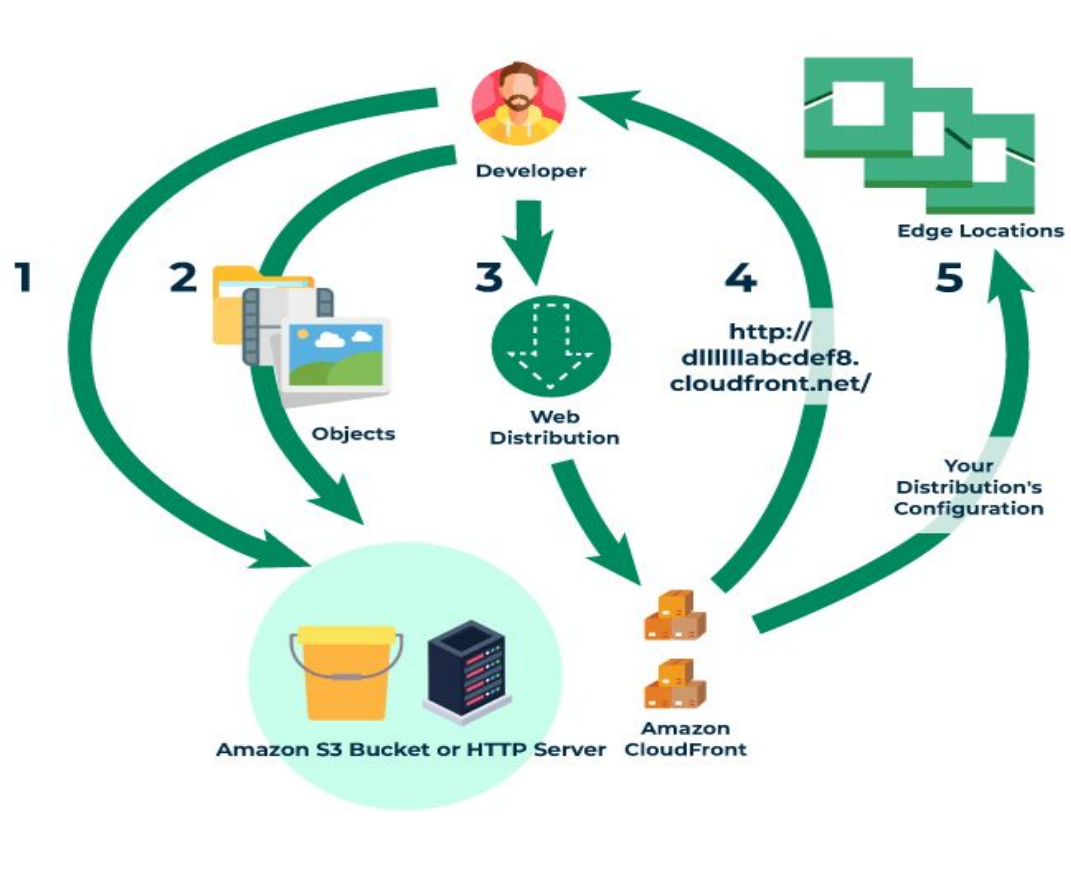
- **Origin:** This is the "source of truth" for your content.
  - It's the server where the original version of your files is stored.
  - An origin can be an Amazon S3 bucket, an Elastic Load Balancer, an EC2 instance, or any custom HTTP/S server (even one on-premises).
- **Edge Location:** This is a data center in a global network, strategically located in major cities around the world (e.g., Tokyo, London, New York, Sydney).
  - CloudFront has hundreds of these.
  - Edge Locations cache (store) copies of your content.
- **Distribution:** This is the core configuration that tells CloudFront *how* to deliver your content.
  - When you create a distribution, you specify your origin(s), set cache rules (like how long to store files), and configure security settings.
  - CloudFront gives you a unique domain name (e.g., d12345.cloudfront.net) for your distribution

Content Delivery Workflow

1. User opens a website
2. Request goes to the nearest CloudFront edge location
3. If cached content exists:
    - CloudFront sends it immediately
4. If not:
    - CloudFront fetches it from the origin server (EC2, S3, Load Balancer, etc.)
    - Stores a cached copy for future users

Benefits

- Fast and global
- Security at the edge
- Highly programmable
- Deeply integrated with AWS
- Cost-effective



Amazon CloudFront pricing

- Data transfer out: Charged for the volume of data transferred out from Amazon CloudFront edge location to the internet or to your origin.
- HTTP(S) requests: Charged for number of HTTP(S) requests.
- Invalidation requests: No additional charge for the first 1,000 paths that are requested for invalidation each month. Thereafter, $0.005 per path that is requested for invalidation.
- Dedicated IP custom SSL: $600 per month for each custom SSL certificate that is associated with one or more CloudFront distributions that use the Dedicated IP version of custom SSL certificate support.

❖ **Simple analogy:**

➢ Imagine a popular bookstore with only one warehouse in one country.

- Without CloudFront:
  - Every customer worldwide waits for delivery from that single warehouse.
- With CloudFront:
  - Small local stores keep copies of popular books in many cities.

➢ So customers get books much faster from the nearest local store instead of waiting for international shipping.

- CloudFront works like global local stores that cache website content near users for faster delivery.

Which One Do I Use?

CloudFront vs. Global Accelerator

Feature	CloudFront	Global Accelerator
Primary Use Case	Caching content and accelerating web applications	Accelerating dynamic traffic and non-HTTP protocols
Supported Protocols	HTTP, HTTPS, WebSocket	**TCP, UDP, HTTP, HTTPS**
Key Feature	**AWS WAF integration** and content caching at 400+ edge locations	**Static Anycast IPs** and instant failover under 60 seconds
Best For	Websites, APIs that benefit from caching, video streaming	Gaming, IoT, VoIP, financial trading applications
Routing Intelligence	Cache-based distribution with geo-restriction capabilities	Performance-based routing with health-check failover
IP Address	Dynamic, changes based on edge location	Two static IPs that never change

Rule of thumb: If the response can be cached, use CloudFront. If every request requires real-time backend processing and you need TCP/UDP support or static IPs, use Global Accelerator.

Terraform

EIU

- Terraform is a tool for building, changing, and versioning infrastructure safely and efficiently. Terraform can manage existing and popular service providers as well as custom in-house solutions.

Key features of Terraform are:

- **Infrastructure as Code**: Infrastructure is described using a high-level configuration syntax.
  a. This allows a blueprint of your datacenter to be versioned and treated as you would any other code. Additionally, infrastructure can be shared and re-used.
- **Execution Plans:** Terraform has a "planning" step where it generates an execution plan.
  a. The execution plan shows what Terraform will do when you call apply. This lets you avoid any surprises when Terraform manipulates infrastructure.
- **Resource Graph:** Terraform builds a graph of all your resources, and parallelizes the creation and modification of any non-dependent resources.
  a. Because of this, Terraform builds infrastructure as efficiently as possible, and operators get insight into dependencies in their infrastructure.
- **Change Automation:** Complex changesets can be applied to your infrastructure with minimal human interaction.
  a. With the previously mentioned execution plan and resource graph, you know exactly what Terraform will change and in what order, avoiding many possible human errors.

Terraform in AWS

Terraform interacts with AWS through the AWS Provider, a plugin that translates HCL code into AWS API calls. The typical workflow involves four main steps:

- Write: Define resources (e.g., EC2 instances, S3 buckets) in .tf files.
- Init: Run terraform init to download the AWS provider and initialize the working directory.
- Plan: Run terraform plan to preview the changes Terraform will make to match your configuration.
- Apply: Run terraform apply to execute the plan and provision the actual resources in your AWS account.

The primary Terraform commands include:

- **init**: Prepares your directory to run other Terraform commands.
- **validate**: Checks if the configuration is valid.
- **plan**: Shows what changes will be made to your infrastructure.
- **apply**: Executes the changes to create or modify your infrastructure.
- **destroy**: Deletes the infrastructure that was previously created.

Terraform Language

```
resource "aws_vpc" "default_vpc"
{
    cidr_block = "172.31.0.0/16"
    tags = {
        Name = "example_vpc"
    }
}
```

Terraform AWS networking setup in Amazon Web Services.

Create Project Folder using AWS CloudShell.

```
mkdir aws-networking-project
cd aws-networking-project
```

These following files one by one should be created.

```
aws-networking-project/
├── provider.tf
├── variables.tf
├── terraform.tfvars
├── vpc.tf
├── subnet.tf
├── networking.tf
├── security.tf
├── ec2.tf
├── outputs.tf
└── versions.tf
```

- Create file nano versions.tf and paste the following code to create version.

```
terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

- Create file provider.tf by command nano provider.tf . Paste the code.

```
provider "aws" {
  region = var.aws_region
}
```

- **Create variables.tf by** nano variables.tf . Paste the following code.

```
variable "aws_region" {
  description = "AWS Region"
  type        = string
}
variable "aws_key_name" {
  description = "AWS Academy key pair"
  type        = string
}
variable "instance_ami" {
  description = "Amazon Linux 2 AMI"
  type        = string
}
```

- **Create terraform.tfvars by** nano terraform.tfvars . Paste the following code and save it.

```
aws_region   = "us-east-1"
aws_key_name = "vockey"
instance_ami = "ami-0c02fb55956c7d316"
```

- **Create vpc.tf** nano vpc.tf . Paste the following code and save it.

```
resource "aws_vpc" "main" {
  cidr_block           = "10.10.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
    Name = "VPC1-Terraform"
  }
}
```

- Create subnet.tf by nano subnet.tf. Copy the code and paste it. After that save it by CTRL +Z. Yes and Enter.

```
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.10.1.0/24"
  availability_zone       = "${var.aws_region}a"
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet"
  }
}
```

```
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.10.2.0/24"
  availability_zone = "${var.aws_region}a"

  tags = {
    Name = "private-subnet"
  } }
```

- Create networking.tf by nano networking.tf

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "IGW1"
  }
}
resource "aws_eip" "nat_eip" {
  domain = "vpc" }
resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public.id
  depends_on = [aws_internet_gateway.igw]
  tags = {
    Name = "NAT1" } }
```

```
resource "aws_route_table" "public_rt" {
vpc_id = aws_vpc.main.id
route {
cidr_block = "0.0.0.0/0"
gateway_id = aws_internet_gateway.igw.id
}
tags = {
Name = "public-rt"
}}
resource "aws_route_table" "private_rt" {
vpc_id = aws_vpc.main.id
route {
cidr_block = "0.0.0.0/0"
nat_gateway_id = aws_nat_gateway.nat.id
}
tags = {
Name = "private-rt"
} }
resource "aws_route_table_association" "public_assoc" {
subnet_id = aws_subnet.public.id
route_table_id = aws_route_table.public_rt.id
}
resource "aws_route_table_association" "private_assoc" {
subnet_id = aws_subnet.private.id
route_table_id = aws_route_table.private_rt.id
}
```

- Create security.tf by nano security.tf

```
resource "aws_security_group" "public_sg" {
  name   = "public-sg"
  vpc_id = aws_vpc.main.id
ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
ingress {
    from_port   = -1
    to_port     = -1
    protocol    = "icmp"
    cidr_blocks = ["10.10.0.0/16"] }
 ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"] }
egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"] } }
```

```
resource "aws_security_group"
"private_sg" {
  name   = "private-sg"
  vpc_id = aws_vpc.main.id
ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["10.10.1.0/24"]
  }
ingress {
    from_port   = -1
    to_port     = -1
    protocol    = "icmp"
    cidr_blocks = ["10.10.0.0/16"]
  }
egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }}
```

- **Create ec2.tf by** nano ec2.tf

```
resource "aws_instance" "public_ec2" {
  ami                    = var.instance_ami
  instance_type          = "t2.micro"
  subnet_id              = aws_subnet.public.id
  key_name               = var.aws_key_name
  vpc_security_group_ids = [aws_security_group.public_sg.id]
  associate_public_ip_address = true
  tags = {
    Name = "EC2-1"
  }
}
resource "aws_instance" "private_ec2" {
  ami                    = var.instance_ami
  instance_type          = "t2.micro"
  subnet_id              = aws_subnet.private.id
  key_name               = var.aws_key_name
  vpc_security_group_ids = [aws_security_group.private_sg.id]
  tags = {
    Name = "EC2-2"
  }
}
```

- Create outputs.tf by nano outputs.tf

```
output "vpc_id" {
  value = aws_vpc.main.id
}
output "public_instance_ip" {
  value = aws_instance.public_ec2.public_ip
}
output "private_instance_ip" {
  value = aws_instance.private_ec2.private_ip
}
```

- Initialize Terraform through command
  - terraform init
- Validate Configuration
  - terraform validate
- Preview Infrastructure
  - terraform plan
- Deploy Infrastructure
  - terraform apply
  - Type yes to confirm
- Verify in AWS Console (VPC, EC2, Route Tables, NAT Gateway)
- Cleanup
  - terraform destroy
  - Yes to confirm

Further reading

- Chapter 3 – Amazon Networking Services (Jaggia et al., 2020, pp. 72–78)
- AWS CDN https://aws.amazon.com/cloudfront/
- CDN service. https://www.geeksforgeeks.org/devops/aws-cloudfront/
- Introduction to Terraform https://www.geeksforgeeks.org/devops/what-is-terraform/
- Github Repository: https://github.com/hashicorp/terraform

Thank you!

End